Assignment 1: Build the Model from Scratch

March 19, 2026

Overview

In Assignment 1, we contain **5 questions** to explore the usage and configuration of supervised learning models. We then apply these concepts to a real-world classification task, carrying out data exploration, preprocessing, and model evaluation techniques.

Instructions

Please submit your solution by **April 9, 11:59 PM.**
Late submissions are not accepted. You can use any third-party libraries or AI tools to assist your work, **but if plagiarism or cheating on assignments, the grade will be 0.**

Submissions consist of two parts:

1. **Code**
    a. Please host your **code on a public GitHub repository** (commit before deadline).
2. **Report.**
    a. Please ensure your public **GitHub link is included** within your report, and submit your **report via E3.**
    b. **File name:** DM_asg1_{studentID}.pdf

Please write down your report in English. The TA will test whether your code can run successfully.

Github Repository: https://github.com/NYCU-Data-Mining/DM2026-Assignment-1

---

1. Linear Regression (20 points):

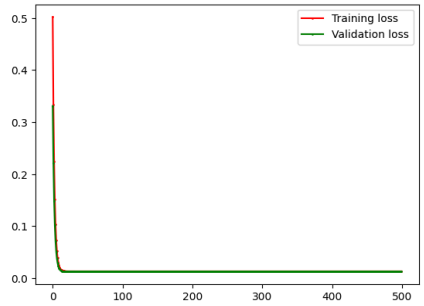
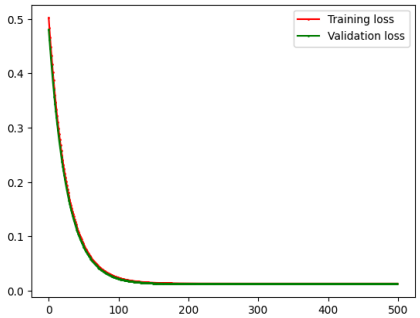
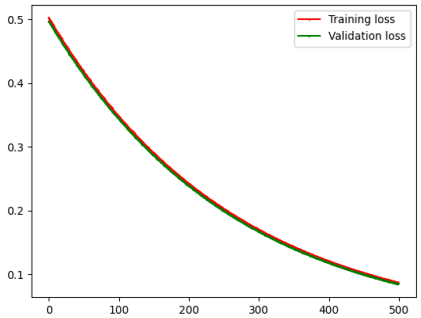
Please refer to **Linear_Regression.ipynb** and file(**metrics.py**) in the model folder. **Do not change random_seed**, as it will affect the evaluation results.

Apply the following questions 1(a) to 1(c) to **all datasets** (linear_data_A, linear_data_B, linear_data_C, and linear_data_D) in the data/linear_data folder.

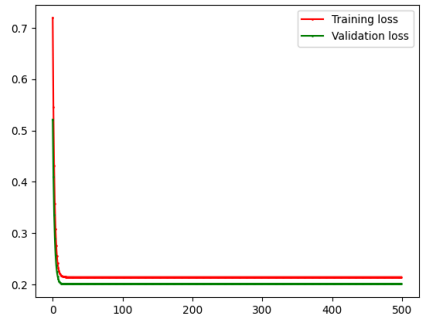
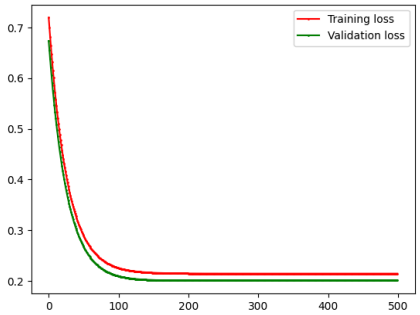
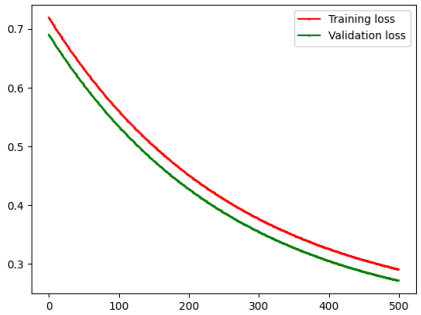
a. **Change the learning rate to the following values, and plot the loss curve for each learning rate:**

$$\text{learning rate} \in \{0.1, 0.01, 0.001\}$$

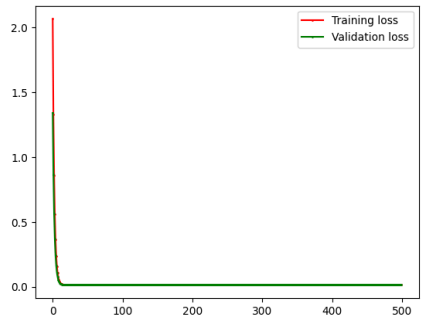
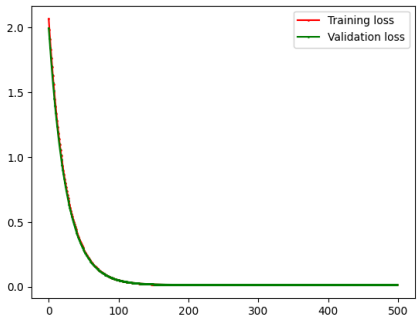
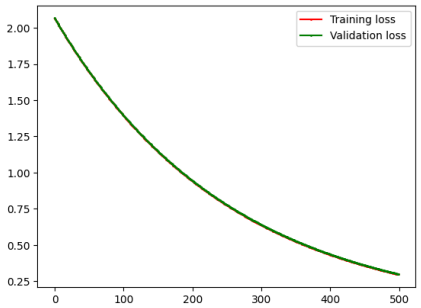
linear_data_A

0.1	0.01	0.001
		

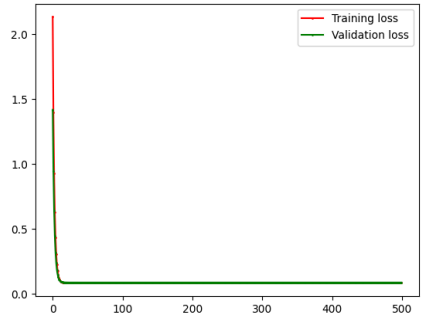
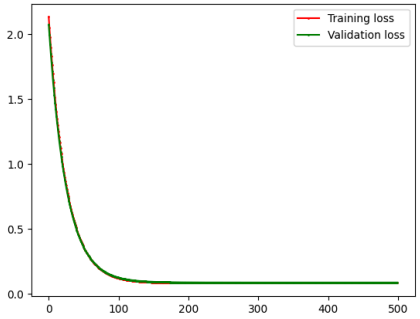
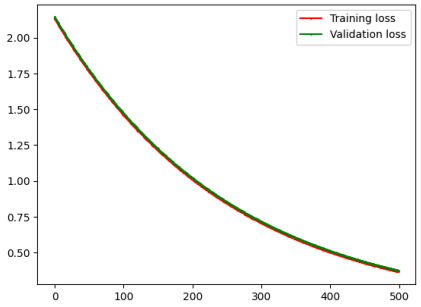
linear_data_B

0.1	0.01	0.001
		

linear_data_C

0.1	0.01	0.001
		

linear_data_D

0.1	0.01	0.001
		

b. Following 1(a), show the evaluations by using the "evaluate_linear_regression" function:

linear_data_A

0.1	0.01	0.001
MSE : 0.0134 MAE : 0.1010 RMSE : 0.1156 R-squared : 0.5690	MSE : 0.0134 MAE : 0.1010 RMSE : 0.1156 R-squared : 0.5690	MSE : 0.0832 MAE : 0.2369 RMSE : 0.2884 R-squared : -1.6836

linear_data_B

0.1	0.01	0.001
MSE : 0.2137 MAE : 0.4038 RMSE : 0.4623 R-squared : 0.0448	MSE : 0.2137 MAE : 0.4038 RMSE : 0.4623 R-squared : 0.0448	MSE : 0.2687 MAE : 0.4404 RMSE : 0.5184 R-squared : -0.2011

linear_data_C

0.1	0.01	0.001
MSE : 0.0132 MAE : 0.0987 RMSE : 0.1148 R-squared : 0.9954	MSE : 0.0132 MAE : 0.0987 RMSE : 0.1148 R-squared : 0.9954	MSE : 0.2852 MAE : 0.4273 RMSE : 0.5341 R-squared : 0.8994

linear_data_D

0.1	0.01	0.001

MSE : 0.0824 MAE : 0.2467 RMSE : 0.2870 R-squared : 0.9716	MSE : 0.0824 MAE : 0.2467 RMSE : 0.2870 R-squared : 0.9716	MSE : 0.3540 MAE : 0.4778 RMSE : 0.5950 R-squared : 0.8780

c. Describe your observations in 1(a) and 1(b).

We can see in 1(a) that the loss is the highest for linear_data_B (when we look at the y-axes, the smallest value is close to 0.2, while for the other examples it is close to 0). And indeed, the difference is confirmed by the 1(b) stats (MSE is used as the loss in the figures).

Plus, we notice that the higher the learning rate, the smaller the loss. This makes sense since the model learns faster with a higher learning rate; thus, it fits the training data faster, which leads to a smaller loss.

2. Logistic Regression (20 points):

Please refer to **Logistic_Regression.ipynb** and file(**metrics.py**) in the model folder. **Do not change random_seed**, as it will affect the evaluation results.

Apply the following questions 2(a) to 2(f) to **all datasets** (logistic_data_A, logistic_data_B, logistic_data_C, and logistic_data_D) in the data/logistic_data folder.

a. Change the learning rate to the following values, and plot the loss curve for each learning rate:

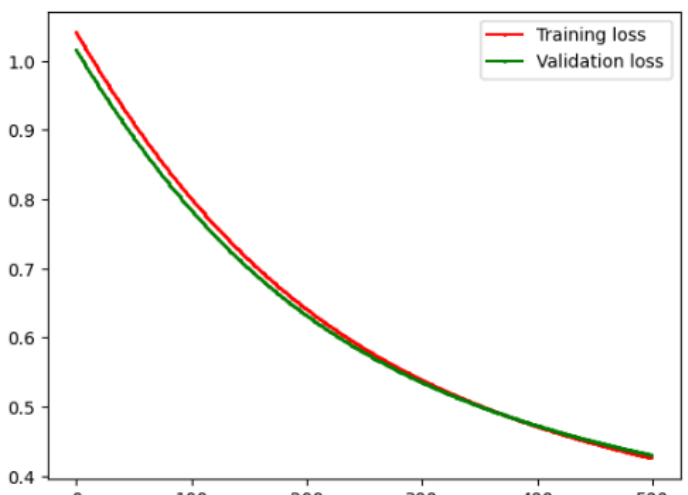
$$\text{learning rate} \in \{0.1, 0.01, 0.001\}$$

[Hint]

- ❖ Change the learning rate by modifying the code shown in the figure on the left (same as 1(a)).
- ❖ Each dataset shows three figures of loss curves (learning rate$\in\{0.1, 0.01, 0.001\}$). The right figure shows an example of plotting the loss curve.

```
# Setting model and starting training
model = LinearModel(dim,False,loss_fn,grad_fn,act_fn)
model.fit(X_train,y_train,lr=0.01,n_iteration =500,val_ratio=0.2)
```

```
20. Training loss: 0.9881638457810773, Val loss:0.9645922922843523
40. Training loss: 0.9363184152246646, Val loss:0.9145933994581312
60. Training loss: 0.8881255625797497, Val loss:0.8681824340076577
80. Training loss: 0.8435013650703489, Val loss:0.8252770470945283
100. Training loss: 0.8023254524923707, Val loss:0.7857578532165119
120. Training loss: 0.76444573392194, Val loss:0.7494732570750171
140. Training loss: 0.7296845906457493, Val loss:0.7162457692441115
160. Training loss: 0.6978459139764438, Val loss:0.6858791790119394
180. Training loss: 0.6687223195489947, Val loss:0.6581659125603253
200. Training loss: 0.6421019406042985, Val loss:0.6328939736163959
220. Training loss: 0.6177743566104512, Val loss:0.6098530110981566
240. Training loss: 0.5955353995655434, Val loss:0.5888392400909019
260. Training loss: 0.5751907540224198, Val loss:0.569659118137967
280. Training loss: 0.5565584019616943, Val loss:0.5521318211243628
300. Training loss: 0.5394700517290799, Val loss:0.536090659945112
320. Training loss: 0.523771734880466, Val loss:0.5213836307216283
340. Training loss: 0.5093237652918465, Val loss:0.5078733052248261
360. Training loss: 0.4960002422502134, Val loss:0.49543625549682435
380. Training loss: 0.4836882535328744, Val loss:0.4839621786917089
400. Training loss: 0.472286903656601, Val loss:0.47335285416404
420. Training loss: 0.4617062619673774, Val loss:0.4635210312543787
440. Training loss: 0.4518662981817047, Val loss:0.45438931672260663
460. Training loss: 0.4426958507480997, Val loss:0.4458891068799368
480. Training loss: 0.4341316561647025, Val loss:0.4379595912834171
```



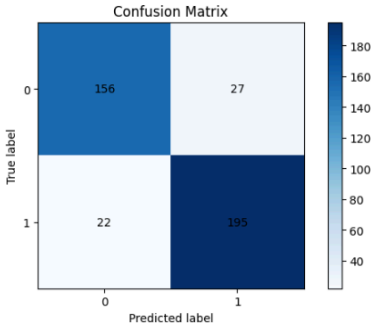
b. Following 2(a), show the evaluations by using the "evaluate_binary_classifier" function.

[Hint]

- ❖ Each dataset shows three evaluation results (learning rate∈{0.1, 0.01, 0.001}).
- ❖ Each evaluation (shown in a screenshot, as the figure on the left) contains the metrics: Accuracy, Precision, Recall, F1 score.
- ❖ Before evaluating, you must complete the TODO sections (as the figure on the right) in model/metrics.py. Please use the pre-imported sklearn package.

```
# Evaluation
# Please complete the evaluate_binary_classifier in the model/metrics.py by the sklearn package.
evaluate_binary_classifier(y_true=y_val,y_pred=y_val_pred,title='Logistic Regression Evaluation')
```

```
Logistic Regression Evaluation
Accuracy  : 0.8775
Precision : 0.8784
Recall    : 0.8986
F1-score  : 0.8884
```




```
# TODO
metrics = {
    'Accuracy': 'TODO: use sklearn.metrics to compute accuracy',
    'Precision': 'TODO: use sklearn.metrics to compute Precision',
    'Recall': 'TODO: use sklearn.metrics to compute Recall',
    'F1-score': 'TODO: use sklearn.metrics to compute F1-score'
}
```

c. Describe your observations in 2(a) and 2(b).

d. Change the number of training iterations to the following values, and plot the loss curve for each setting (using a fixed learning rate of 0.01**):**

n_iteration∈{500, 1000, 1500}

[Hint]

- ❖ Each dataset shows three figures of loss curves (n_iteration∈{500, 1000, 1500}), presented in a format similar to that in 2(a).

e. Following 2(d), show the evaluations by using the "evaluate_binary_classifier" function.

[Hint]

- Each dataset shows three evaluation results (n_iteration∈{500, 1000, 1500}).
- Each evaluation (shown in a screenshot) contains the metrics: Accuracy, Precision, Recall, F1 score, presented in a format similar to that in 2(b).

f. Describe your observations in 2(d) and 2(e).

3. Data Preprocessing (20 points):

Please refer to **Real_World_Classification.ipynb**. **Do not change random_seed**, as it will affect the evaluation results. NYCU_Iris.csv is in the data folder.

a. Report the **median** and **standard deviation** for columns with missing values.

b. Write the code to **replace the missing values** using "**Nearest Neighbors Imputation**" for columns with missing values. Report the **median** and **standard deviation** of those columns. Compare your result with 3(a) and **explain your observation**.

[Hint]

- sklearn.impute

```
# TODO: Replace the missing values using "Nearest Neighbors Imputation"
# ---------- Start your code below ----------


# ---------- -------------------- ----------
```

4. Data Exploration (20 points):

Please continue using **Real_World_Classification.ipynb**. Based on the preprocessed data from Task 3, proceed with the data exploration. Do not change the random_seed to ensure consistency in evaluation results.

a. Plot the histogram of the column "PetalWidthCm" in "NYCU_Iris.csv". Please **describe your observations** for the histogram.

b. **List the feature** (excluding "PetalWidthCm", "Species", and "Id") that has the **largest positive correlation** (Pearson correlation coefficient) **with the "PetalWidthCm"** column.

[Hint]

- Pearson correlation: sklearn.feature_selection.r_regression

c. **Report the top 5 features** with the **strongest negative** correlations with the 'PetalWidthCm' column.

d. **Plot the boxplot** for all features found in 4(b) and 4(c).

5. Regularization (20 points):

Please continue using **Real_World_Classification.ipynb**. Based on the preprocessed data from Task 3, proceed with the data exploration. Do not change the random_seed to ensure consistency in evaluation results.

a. **L2 Regularization.** Please run Real_World_Classification.ipynb to compare the performance between no regularization (current code) and L2 regularization. For L2 regularization, set the regularization parameter (reg_lambda in the code) to 0.01, 1, and 100, respectively. Then, **plot the loss curves** for all settings:

(1) No regularization
(2) L2 (reg_lambda=0.01)
(3) L2 (reg_lambda=1)
(4) L2 (reg_lambda=100)

[Hint]

- Four figures in total, presented in a format similar to that in 2(a)
- To use L2 regularization, please set the reg_type='l2' and reg_lambda value in the fit() function. For example: model = LinearModel (..., reg_type='l2', reg_lambda=0.01)

b. **Following 5(a), show the testing results by using the "evaluate_binary_classifier" function.**

[Hint]

- Show two testing results (no regularization and L2).
- Each testing result (shown in a screenshot) contains the metrics: Accuracy, Precision, Recall, F1 score, presented in a format similar to that in 2(b).

c. Describe your observations in 5(a) and 5(b).

Reference

1. https://github.com/anhquan0412/basic_model_scratch/tree/master
2. https://scikit-learn.org/stable/api/sklearn.metrics.html
3. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/fork-a-repo

FAQ

Below are some common questions and answers. If you have any other questions, please feel free to contact the TA at nycu.dm.ta@gmail.com.

1. How to download the code from github?

1. Fork the repository: Click the "Fork" button at the top right to create a copy under your own GitHub account.
2. Clone to your local machine:

   Open your terminal and run: git clone https://github.com/XXX/XXX.git

If this is your first time contributing to a project, you can follow this official tutorial: Fork a repo - GitHub Docs.

2. How to upload the code to your github repo?

Once you have finished making changes to your code locally, you can upload them to your GitHub repository by following these steps:

```shell
# If you used 'git clone', you can skip this part.
git init
git remote add origin <Your_Repository_URL>

# Stage and Commit
git add .
git commit -m "assignment1"

# Upload
git push origin main
```

If you need more details on managing files, you can refer to this guide: Adding a file to a repository - GitHub Docs